

Multi-Agent Systems



11 Nov 2025 – Lecture 4 – Prolog Recursion



Recursion

- Recursion in procedural / object-oriented programming:
 - A method of solving a problem where the solution depends on solutions to smaller instances of the same problem
 - A recursive definition is defined in terms of itself
 - Important to include termination condition
- Recursion in prolog
 - Compact form to define predicates
 - Let the computer do the work → reasoning
 - Danger of non-termination

Reachable

A predicate *pred2* is reachable from predicate *pred1* in a program P if:

1. There is a rule in P the head of which uses *pred1* and the body of which contains *pred2*; or

2. there is a predicate *pred3* such that *pred3* is reachable from *pred1* in P, and *pred2* is reachable from *pred3* in P.

```
pred1(X) :- pred2(X).
```

```
pred1(X) :- pred3(X).
```

```
pred3(X) :- pred2(X).
```

Recursion

A predicate *pred* is defined **recursively** if there is at least one rule in the definition of *pred* in *P* from which *pred* itself is reachable.

Ex: The predicate *p/1* with $p(X) \text{ :- } p(X), q(X).$ is defined recursively.

The following program is also a recursive (indirect) definition of *p/1*:

- $p(X) \text{ :- } q(X), r(X,X).$
- $r(X,X) \text{ :- } p(X).$

Example

We would like to define a descendent relation, using predicates `child/2` and `descendent/2`, between Paul and Sara:

```
child(paul, ann).  
child(ann, sara).  
descendent(X,Y) :- child(X,Y).  
descendent(X,Y) :- child(X,Z), child(Z,Y).
```

We can now derive `descendent(paul, sara)`.

Example (continued)

What if we add the fact: `child(sara, frank)`.

```
child(paul, ann).  
child(ann, sara).  
child(sara, frank).  
descendent(X,Y) :- child(X,Y).  
descendent(X,Y) :- child(X,Z), child(Z,Y).  
descendent(X,Y) :- child(X, W), child(W, Z), child(Z, Y).
```

Example (continued)

What if we add another fact, etc.?

Base clause

`descendent(X, Y) :- child(X, Y).`

Recursive clause

`descendent(X, Y) :- child(X, Z), descendent(Z, Y).`

Example (continued)

What if we add the fact: `child(sara, frank)`.

```
child(paul, ann).  
child(ann, sara).  
child(sara, frank).  
descendent(X,Y) :- child(X,Y).  
descendent(X, Y) :- child(X, Z), descendent(Z, Y).
```

Let's try ?- `descendent(paul, frank)`.

Example – order of returns

Consider the following program and query:

```
child(paul, ann).  
child(ann, sara).  
child(sara, frank).  
descendent(X,Y) :- child(X,Y).  
descendent(X, Y) :- child(X, Z), descendent(Z, Y).
```

```
?- descend(A,B).
```

Recursion: rules of thumb

- Recursion is powerful and dangerous.
- Try to **avoid left-recursion**. It is usually better to make the recursive call as late as possible (more variables will be instantiated).
- Recursive call **as far as possible to the right** of a rule.
- Rule and predicate ordering is important.
- Try to put **base clause before recursive clause**.

Non-Termination

Recursion is powerful but can yield **non-terminating** behavior.

Consider the program and query:

```
s(a).  
r(b).  
p(X) :- r(X).  
p(X) :- s(X).  
p(X) :- t(X).  
t(X) :- t(X).
```

```
?- p(a).
```

The query yields a successful and an infinite branch (in that order).

Lists in summary

- **[Head | Tail]** is a list with head Head and tail Tail.
- the **head** of a list is a **term**,
- the **tail** of a list **is a list** itself.
- empty list (`[]`) has neither tail nor head.

- Recurring down a list is a very important technique when programming with Prolog.

Example of recursion and lists: Checking membership



A program that checks whether a term is a member of a list.

`member(X, L)` should succeed if `X` is contained in `L`.

Idea of a recursive definition:

- `X` is the head of `L`, or
- `X` is a member of the tail of `L`

Member definition

Base clause:

`member(X, [X|_]).`

Recursive clause:

`member(X, [_|Tail]) :- member(X, Tail).`

```
[trace] ?- X = [1,2,3,4], m(4,X).
  Call: (11) _12042=[1, 2, 3, 4] ? creep
  Exit: (11) [1, 2, 3, 4]=[1, 2, 3, 4] ? creep
  Call: (11) m(4, [1, 2, 3, 4]) ? creep
  Call: (12) m(4, [2, 3, 4]) ? creep
  Call: (13) m(4, [3, 4]) ? creep
  Call: (14) m(4, [4]) ? creep
  Exit: (14) m(4, [4]) ? creep
  Exit: (13) m(4, [3, 4]) ? creep
  Exit: (12) m(4, [2, 3, 4]) ? creep
  Exit: (11) m(4, [1, 2, 3, 4]) ? creep
X = [1, 2, 3, 4] ;
```

What does `?- X = [1,2,3,4], member(4, X)` return?

The predicate member/2 is built-in in Prolog.

List Programming in Prolog



Recurring down a list is a very important technique when programming with Prolog.

Assignment:

Define a predicate `aEqualB/2` which takes two lists as input and succeeds iff the lists have same lengths; the former consists only of a's; and the latter only of b's.

```
aEqualB([],[]).
```

```
aEqualB([a | L1],[b | L2]) :- aEqualB(L1,L2).
```

Think of the intended condition of the two heads at any position of the lists

Example: appending lists

We define a predicate `append/3` with `append(X, Y, Z)` is true if list `X` combined with list `Y` is list `Z`.

1. The first list empty: then `Z` equals the second list.
2. Else, the first element of first list, must be the first element of `Z`, and
3. Then the `append` predicate is applied on the remainder of the lists recursively.

```
append([], X, X).
```

```
append([X|Tail1], Y, [X|Tail2]) :- append(Tail1, Y, Tail2).
```

Search tree for append/3

?- append([a,b],[c,d], L).

\ L = _G4310

append([a,b],[c,d],_G4310)

/ \ _G4310 = [a|_G4392]

no ?- append([b],[c,d],_G4392).

/ \ _G4392 = [b|_G4395]

no ?- append([], [c,d],_G4395).

/ \ _G4395 = [c,d]

L = [a,b,c,d]. No

append([], X, X).

append([X|Y], Z, [X|W]) :- append(Y, Z, W).

Tracing append/3

`append([], X, X).`

`append([X|Y], Z, [X|W]) :- append(Y, Z, W).`

The trace for query `?- append([a,b], [c,d], L).` looks as follows:

- 1.Call: (6) `append([a, b], [c, d], _G4310) ? creep`
 - 2.Unify: (6) `append([a, b], [c, d], [a| _G4392])`
 - 3.Call: (7) `append([b], [c, d], _G4392) ? creep`
 - 4.Unify: (7) `append([b], [c, d], [b| _G4395])`
 - 5.Call: (8) `append([], [c, d], _G4395) ? creep`
 - 6.Unify: (8) `append([], [c, d], [c, d])`
 - 7.Exit: (8) `append([], [c, d], [c, d]) ? creep`
 - 8.Exit: (7) `append([b], [c, d], [b, c, d]) ? creep`
 - 9.Exit: (6) `append([a, b], [c, d], [a, b, c, d]) ? creep`
- `L = [a, b, c, d].`

Search table for append/3

query ?- append([a,b], [c,d], L).

append([], X, X).

append([X|Y], Z, [X|W]) :- append(Y, Z, W).

	Query				
Level in tree	First term	Second Term	Third term	Clause matched	Instantiations
	[a,b]	[c,d]	L		L = [a,b,c,d]

Using append/3

`append([], X, X).`

`append([X|Tail1], Y, [X|Tail2]) :- append(Tail1, Y, Tail2).`

The predicate can be used in different ways.

Explain the use of the following queries:

1.?- `append([a, b], [c, d], X).`

2.?- `append([a, 2], [b, 3], [a, 2, b, b]).`

3.?- `append(X, Y, [a, b, c, d]).`

Using append/3 to split a list

`append([ ], L, L).`

`append([H | L1], L2, [H | L3]) :- append(L1, L2, L3).`

Splitting up a list:

`?- append(X,Y, [a,b,c,d]).`

<code>X=[]</code>	<code>Y=[a,b,c,d];</code>
--------------------	---------------------------

<code>X=[a]</code>	<code>Y=[b,c,d];</code>
--------------------	-------------------------

<code>X=[a,b]</code>	<code>Y=[c,d];</code>
----------------------	-----------------------

<code>X=[a,b,c]</code>	<code>Y=[d];</code>
------------------------	---------------------

<code>X=[a,b,c,d]</code>	<code>Y=[];</code>
--------------------------	---------------------

`no`

Example: sublists

These basic predicates can be used as building blocks for more complex predicates.

Define a predicate `sublist(S,L)` which succeeds if `S` is a sublist of `L`. Use `append/3` to define this predicate.

1. `L` can be decomposed into two lists `L1` and `L2`, and
2. `L2` can be decomposed into `S` and another list `L3`.

```
sublist(S,L) :- append(_ ,L2,L), append(S, _ ,L2).
```

Exercise: Define member by append

Define member/2 by means of append/3.

There is a single clause solution:

```
member(X,L) :- append( _ , [X| _ ], L).
```

Exercise: Last element of a list

Define predicate `last/2` which is true if we have the last element of the list, e.g. `last(3, [1, 2, 3])`.

The base clause is given:

`last(X, [X]).`

Recursive clause:

`last(X, [_|T]) :- last(X, T).`

Exercise (same elements in list)

Write a predicate `allSame(L)`, where `L` is a list, which is true if all elements in `L` are identical.

```
allSame([]).
```

```
allSame([X]).
```

```
allSame([H1 | [H2 | T]]) :- H1=H2, allSame([H2 | T]).
```

This can further be simplified:

```
allSame([]).
```

```
allSame([_]).
```

```
allSame([X, X | T]) :- allSame([X | T]).
```

Exercise: List deletion (and insertion)

`del(X,L,D)` succeeds if D is L with one X being removed

1.If X is head of $L=[X|Tail]$ then D can be Tail.

`del(X,[X|Tail],Tail).`

2.Else, consider the lists' tails:

`del(X,[Y|Tail1],[Y|Tail2]) :- del(X,Tail1,Tail2).`

We can easily define a predicate `insert/3`:

- `insert(X, L, Ins) :- del(X, Ins, L).`

Yet another elegant definition of `member/2`:

- `member(X,L) :- del(X, L, _).`

```
[trace] ?- del(d,[a,b,c,d],D).
Call: (10) del(d, [a, b, c, d], _19550) ? creep
Call: (11) del(d, [b, c, d], _19820) ? creep
Call: (12) del(d, [c, d], _19826) ? creep
Call: (13) del(d, [d], _19832) ? creep
Exit: (13) del(d, [d], []) ? creep
Exit: (12) del(d, [c, d], [c]) ? creep
Exit: (11) del(d, [b, c, d], [b, c]) ? creep
Exit: (10) del(d, [a, b, c, d], [a, b, c]) ? creep
D = [a, b, c] ;
```

```
[trace] ?- del(b,[a,b,c,d],D).
Call: (10) del(b, [a, b, c, d], _20114) ? creep
Call: (11) del(b, [b, c, d], _20382) ? creep
Exit: (11) del(b, [b, c, d], [c, d]) ? creep
Exit: (10) del(b, [a, b, c, d], [a, c, d]) ? creep
D = [a, c, d] .
```

```
[debug] ?- del(d,Ins,[a,b,c]).
Ins = [d, a, b, c] ;
Ins = [a, d, b, c] ;
Ins = [a, b, d, c] ;
Ins = [a, b, c, d] ;
false.
```

Example: length of a list

Define a predicate `length/2` which computes the length of a list:

```
% base case
```

```
length([], 0).
```

```
% recursive clause
```

```
length([_ | Tail], N) :- length(Tail, P), N is P + 1.
```

Note: left recursion...

but with smaller list!

Search tree for length /2



?- lenght([a,b,c], N).

/ \

no ?- lenght([b,c],N1), N is N1 + 1.

/ \

no ?- lenght([c], N2), N1 is N2+1,
N is N1+1.

/ \

no ?- lenght([], N3), N2 is N3+1,
N1 is N2+1, N is N1 + 1.

/

N3=0, N2=1, N1=2, N=3

\

no

length([],0).

length([_ | Tail], N) :- length(Tail, P), N is P + 1.

Exercise: flatten list

`flatten_list/2` flattens a nested list, removing all nested lists and combining elements into a single list. You can use `append/3`.

```
% Base clause: an empty list flattens to an empty list.
```

```
flatten_list([], []).
```

```
% Recursive clause 1: if the head is a list, flatten it and concatenate it with the flattened tail.
```

```
flatten_list([Head|Tail], FlatList) :-
    is_list(Head),
    flatten_list(Head, FlatHead),
    flatten_list(Tail, FlatTail),
    append(FlatHead, FlatTail, FlatList).
```

```
% Recursive clause 2: if the head is not a list, add it to the flattened tail.
```

[illegible]

Example: duplicating elements in list

Goal

?- duplicateList([a, b, c], Duplicated).
Duplicated = [a, a, b, b, c, c].

?- duplicateList([1, 2, 3, 4], Duplicated).
Duplicated = [1, 1, 2, 2, 3, 3, 4, 4].

% Base clause: an empty list duplicates to an empty list.
duplicateList([], []).

% Recursive clause: duplicate the head of the list, and then recursively duplicate the tail.
duplicateList([Head|Tail], [DupHead1, DupHead2 | DupTail]) :-
 duplicateElement(Head, [DupHead1, DupHead2]), % Duplicate the head
 duplicateList(Tail, DupTail). % Recursively duplicate the tail.

% Helper predicate to duplicate a single element.
duplicateElement(Element, [Element, Element]).

Exercise: count vowels in a list

?- nr_vowel([],X).

X = 0

?- nr_vowel([a,r,e,d,i],X).

X = 3

?- nr_vowel([m,r],X).

X = 0

?- nr_vowel([s,e,e,d],X).

X = 2

Exercise: count vowels in a list

```
% auxiliary predicate
vowel(X):- member(X,[a,e,i,o,u]).

% base case
nr_vowel([],0).

% recursive clauses
nr_vowel([X|T],N) :-
    vowel(X), nr_vowel(T,N1), N is N1+1.
nr_vowel([X|T],N):-
    \+ vowel(X), nr_vowel(T,N).
```

Planning:

- Thursday November 13
 - Practicum assignment 2
- Tuesday November 18
 - Lecture: Intro MARBEL and Cognitive States
 - Prof.dr. Koen Hindriks
 - Preparation: MARBEL programming guide Ch. 1, 2, 3
- Monday November 24
 - Deadline assignment 2